

ELI — Orchestration And Agents

ELI v2.4.82

September 2026

Contents

ELI Orchestration & Agents — Full Topology	1
Two agent stacks	1
Planning artifacts	3
Where it is still weak	3
Habits reach the chat path	4
Custom agents have a specification and a trust chain	4

ELI Orchestration & Agents — Full Topology

Updated for v2.4.72. All CHAT modes run the orchestrator at scaled depth; retrieval is unified in `eli/memory/retrieval.py`; stage 12 learning is centralised in `learning_coordinator.py`; the direct-versus-synthesise gate (`_deterministic_direct_payload_actions`) has been audited against what the executor handlers actually return.

Read-only reference; nothing here changes behaviour.

Source files: -`eli/cognition/orchestrator.py`: the orchestrator (12-stage pipeline) -`eli/cognition/agent_bus.py`: the parallel 15-agent specialist bus -`eli/memory/retrieval.py`: shared turn retrieval (bus and orchestrator) -`eli/cognition/learning_coordinator.py`: stage 12 `finalize_turn()` -`eli/kernel/pipeline_trace.py`: canonical S01-S12 logging -`eli/kernel/engine.py`: wiring and the dispatch gate -`eli/execution/execution_planner.py`: the typed plan model

Two agent stacks

ELI has **one primary cognition path** for CHAT and a **parallel specialist bus** that the orchestrator composes (or that the engine falls back to).

1. AgentOrchestrator (orchestrator.py)

The 12-stage cognitive pipeline. The engine calls it for every CHAT mode, Quick through Expert, at a depth chosen by `mode_orchestrator_depth()` and `orchestrator_planner_mode()` (`reasoning_modes.py`). Components:

- **PlannerAgent.plan_retrieval()** produces a mode-aware retrieval plan:
 - fast: keyword only, no FAISS or RAG, knowledge graph only for identity, one ReAct iteration, no HyDE;
 - balanced (default): keyword, semantic and knowledge graph, RAG for document queries, three ReAct iterations;
 - deep: everything, large budgets, full HyDE, three ReAct iterations. The orchestrator then attaches the time window found in the question (`query_planner.parse_window`) to the plan.

- **OrchestratorMemoryAgent** delegates to `retrieve_for_turn()` in `memory/retrieval.py` (shared with the bus): HyDE expansion, keyword and FTS5, FAISS semantic, document RAG and knowledge graph, hybrid_merge, then a heuristic rerank (`rerank_candidates`). Sequential by design: the `llama.cpp` embedder is not thread-safe.
- **ExecutorAgent** is a thin wrapper over `executor_enhanced.execute`.

Flow inside `AgentOrchestrator.run()`:

- **Non-CHAT actions:** dispatch the AgentBus for specialist evidence, then a **ReAct observation loop**: run the executor, ask the loaded model for ANSWER or TOOL:<action> <args>, chain to the next tool and accumulate observations. One iteration in fast mode, up to three otherwise. The proposed tool is validated against the executor’s SUPPORTED_ACTIONS (206 actions) and the loop stops on an unknown action; `intent["args"]` are merged rather than overwritten. For “grounded synthesis” actions the observations are assembled into context and passed to the model; for direct actions the executor result is returned as is.
- **CHAT:** planner → shared retrieval → `dispatch_specialists()` (mode-aware fan-out; memory skipped when already prefetched) → context assembly and a retrieval-diagnostics block → persona handoff → generation. Private reasoning modes (Normal, Advanced, Research, Expert) hand off to `engine._run_chat_reasoning_loop`. The bus is composed on the CHAT path and is not bypassed in Quick mode.

The direct-versus-synthesis decision is a set membership test in `eli/kernel/engine.py`:

- `_deterministic_direct_payload_actions` (191 entries): the executor’s content or response is returned verbatim in quick mode; in other modes such an action may be re-narrated by `_compact_grounded_synthesis()` (constrained to quote from evidence, validated against it, falling back to the raw evidence).
- `_verbatim_always_actions` (16 entries): verbatim in every mode.
- `eli/runtime/response_contracts.py::_QUICK_ACTIONS` (8 entries) only feeds the prompt header and never decides verbatim versus synthesis.

The 187 routable actions were each checked against their executor handler’s real return shape (not assumed from the name): raw file reads, shell output, MCP results, transcriptions and OCR text, confirmations and status reports are verbatim; `ANALYZE_IMAGE`, `ANALYZE_PDF[_FOLDER]`, `SCREEN_READ_ANALYZE`, `DATA_FABRICATOR`, `GENERATE_PROJECT`, `SEQUENCE` and `MULTI_COMMAND` already return finished text and are verbatim too. Excluded on purpose, each with a reason: `FIX_FILE` and `GENERATE_SCRIPT` (their content is a JSON event for the GUI), `RUN_TESTS` (meant to be summarised), `CHAT` (the model call itself), `SHOW_DIFF` (routes to `chat()`), `WEB_SEARCH` (results are evidence, not the answer), `CODE_SOLVE` (generative, goes to the coding agent), `EXECUTE_GOAL` and `NOOP` (no single handler to verify). `tests/test_deterministic_actions_cover_status_and_read_file.py` holds the audited lists and a completeness test that fails if a routable action lands in neither set nor the exclusion list.

2. AgentBus: the parallel 15-agent fan-out (`agent_bus.py`)

15 agents in `_ALL_AGENTS`, each a `_BaseAgent` subclass with a name and `timeout_s`:

#	name	timeout_s	accesses
1	memory	5.0	SQLite, FTS5, FAISS; self-gates
2	system	8.0	direct action execution (<code>SYSTEM_ACTIONS</code>)
3	habit	3.0	user_patterns, learned and detected habits
4	self_improvement	3.0	failures, improvement proposals, corrections
5	proactive	3.0	suggestion and anticipation
6	frontier	5.0	frontier and awareness model
7	plugin	6.0	plugin registry (<code>PLUGIN_ACTIONS</code>)
8	capability	6.0	capability manifest
9	voice	5.0	TTS and voice subsystem
10	orchestrator	3.0	bus-level planner; emits a plan dict only

#	name	timeout_s	accesses
11	file_code	4.0	source tree and code introspection
12	reflection	4.0	reflection log and insights
13	introspection	4.0	runtime and cognition self-inspection
14	knowledge_graph	3.0	entity and relation graph (runs after memory)
15	critic	2.0	verifies retriever output (runs after memory, file_code, knowledge_graph, system)

SpecAgent instances (user-defined specifications) and custom agents register on top of these.

Execution (`AgentBus.dispatch()`):

- **Selective fan-out:** tiny filler chat $\rightarrow$ {memory, orchestrator}; non-chat $\rightarrow$ `_select_agents_for_intent` (a minimal set from a keyword and action ladder); plain CHAT $\rightarrow$ broad fan-out across all enabled agents.
- **Dependency DAG:** `_AGENT_DEPENDENCIES` orders agents in topological layers, running the agents of a layer in parallel; `knowledge_graph` waits for memory, and `critic` waits for its four retrievers. If the DAG fails the bus falls back to a flat parallel run.
- **Per-agent hard timeout:** a timeout or exception becomes a failed `AgentResult` and never blocks the response.
- **Aggregation** (`_aggregate_confidence`): per-agent contribution = evidence quality $\times$ evidence density $\times$ a calibration learned per (agent, action); a single-agent cap; an empty-bus ceiling; a corroboration bonus when several agents contribute.

When each runs

Situation	Path
CHAT (Quick, Normal, Advanced, Research, Expert)	AgentOrchestrator at mode depth $\rightarrow$ shared retrieval $\rightarrow$ <code>dispatch_specialists()</code>
Non-CHAT action (any mode)	AgentOrchestrator $\rightarrow$ bus and ReAct loop
Orchestrator returns None or raises	falls back to AgentBus.dispatch() directly
Phatic or ultra-short filler (engine heuristic)	may use a lean bus profile inside the orchestrator

Stage 12 side effects (store the turn, publish meta, `_learn_from_result`) run through `learning_coordinator.finalize_turn()`, one entry point for every CHAT exit.

Planning artifacts

1. **ReAct loop:** the only planner that sequences execution (tool $\rightarrow$ observe $\rightarrow$ decide $\rightarrow$ next).
2. **PlannerAgent.plan_retrieval:** plans retrieval budgets, not actions; used every CHAT turn.
3. **Bus OrchestratorAgent plan:** an `orchestrator_plan` dict stored in the trace for display, persona handoff and status; never executed.
4. **execution_planner.build_execution_plan** (`ExecutionPlan`, `PlanStep`): the canonical typed plan. `AgentBus.dispatch` builds it each turn, injecting the result of `_select_agents_for_intent` as `agent_profile`, and drives the active agents through it; it is exposed as `DispatchResult.execution_plan`. `EXECUTE_GOAL` also builds a plan through it.

Where it is still weak

1. **Bus agents cannot consume each other's output** beyond the declared DAG edges; only `knowledge_graph` and `critic` have dependencies.

2. **Planning is only partly consolidated.** The engine's `_build_runtime_orchestrator_plan` (a rich stage dict) and the bus `OrchestratorAgent` plan coexist with the typed `ExecutionPlan`; only the `ReAct` loop executes a sequence.
3. **Timeouts do not cancel work.** `future.result(timeout)` only stops waiting; a timed-out write-capable agent (memory, habit) can still land a late database write.
4. **Confidence is coupled to a fixed evidence-key schema.** `_evidence_density` counts known keys; a custom agent returning useful prose under an unknown key contributes zero.
5. **No early exit for direct actions; failures are debug-only.** The bus waits for the slowest selected agent even when a direct `action_result` exists, and timeouts log at debug level with no health surface, although an `agent_metrics` table exists.

Fixed and worth knowing: `_apply_runtime_policy_timeouts()` is applied to built-ins and again after `_load_custom_agents()`, so custom agents get hardware-adapted timeouts too.

Highest leverage now: an agent-health surface built from `agent_metrics` and `pipeline_trace`, and a cooperative cancel token for write-capable agents.

Habits reach the chat path

The bus `HabitAgent` reads both `get_habit_rules()` and `get_detected_habits()` (the habits table the proactive daemon fills), emits a summary and detected habits, and `DispatchResult.to_context_block` renders the summary into the chat context. Goal autogenesis (`planning/goal_autogenesis.py`) feeds the autonomy and goal-tick stack from ELI's own signals; see `runtime_planning_world.md`.

Custom agents have a specification and a trust chain

A custom agent was once a `.py` file dropped in a directory with a name, a `timeout_s` and an optional free-text persona. It is now **data, not code**.

AgentSpec (`eli/cognition/agent_spec.py`)

Field	Required	Purpose
<code>objective</code>	yes	one sentence on what this agent is responsible for
<code>system_prompt</code>	yes	the instruction the model receives
<code>triggers</code>	at least 1	keyword, regex, action, always
<code>success_criteria</code>	at least 1	runnable checks: contains, not_contains, regex, min/max length, non_empty, is_json
<code>examples</code>	no	input and expected checks, so the agent can be tested before going live
<code>permissions</code>	no	capabilities from the plugin vocabulary, gated at run time

`validate()` refuses vagueness: an objective under 25 characters or matching a placeholder list (`todo`, `does stuff`, `helper`, ...) is rejected, as is a system prompt under 40 characters. An agent with no trigger is refused because it would never run; one with no success criterion is refused because nothing could tell whether it worked; an always trigger is allowed with a warning about latency. `evaluate(output)` runs the criteria and returns a score, which is the measure the wizard's test step uses and the confidence `SpecAgent` reports. `content_hash()` excludes created and enabled, so re-saving a spec does not invalidate a trust grant while a real edit does.

SpecAgent (agent_bus.py)

Runs a spec: check triggers, check declared permissions, call the local model with the spec's system prompt, score the output against the criteria. An agent that fails its own success test contributes nothing. Because a spec executes no arbitrary code, spec agents need no trust grant; the hash, scan and provenance chain applies only to code agents.

Loader

- `_custom_agent_dirs()` puts the data directory first (`<data>/agents/custom`), so a created agent has somewhere valid to live on any install.
- Trust is checked through `agent_trust.inspect()` (see `security.md`), which reports why an agent was not loaded.
- `agent_load_report()` records the per-agent outcome so the GUI can show the reason.
- `reload_custom_agents()` re-scans specs and code without a restart and replaces previously registered spec agents.